Learning Combinatorial Distance Heuristics:

Optimizer and Hyperparameter Effects on a $2{\times}2{\times}2$ Rubik's Cube Solver

Cole McGuire
Colorado School of Mines
MATH 598B: Numerical Optimization

Spring 2026

Abstract

We study the numerical optimization of a small transformer trained to approximate BFS optimal distance on the $2{\times}2{\times}2$ Rubik's cube—a combinatorial domain where ground-truth optimal distances are exactly computable. We compare three first-order optimizers (AdamW, Adam, SGD with momentum) and three learning rate schedules (cosine annealing, constant, step decay), conduct a hyperparameter sensitivity analysis over model width and weight decay, analyze the quality of the learned distance heuristic relative to BFS ground truth, and visualize the loss landscape via filter-normalized random directions [1]. Over three random seeds, AdamW and Adam converge to $78.3{\pm}1.0\%$ and $78.9{\pm}0.2\%$, respectively, while SGD trails by $\sim$4.5 pp (74.4%, std $<$0.1 pp); cosine annealing outperforms step and constant schedules by $\sim$1 pp; and weight decay provides no benefit on this clean combinatorial task. The learned heuristic achieves perfect accuracy at distances 0–2 but degrades sharply for harder states, overestimating 12.5% of the time and therefore violating the admissibility condition required for A* optimality guarantees. A two-dimensional loss-landscape visualization reveals a broad, flat basin around the trained solution, consistent with the low hyperparameter sensitivity observed across experiments.

1 Introduction

Training a deep neural network is fundamentally a non-convex numerical optimization problem. The choice of optimizer and learning rate schedule determines not only whether the model converges, but how quickly, to what quality of solution, and with what sensitivity to hyperparameters. Understanding these dynamics is difficult in general: loss surfaces are intractable, ground-truth optimal performance is unknown, and empirical comparisons on large benchmarks conflate optimizer effects with task difficulty.

The $2{\times}2{\times}2$ Rubik's cube offers a rare controlled setting. Its full state space—roughly 3.7 million reachable positions—is small enough that the true minimum number of moves to solve any state (the *optimal distance*) can be computed exactly via BFS. God's number for the $2{\times}2{\times}2$ is 11 moves in the half-turn metric [2]. A neural network trained to predict this distance is therefore learning to approximate a *known* function with structured geometry: distance is 0 at the solved state, changes by at most one under any legal face turn, and is symmetric under rotational equivalences of the cube. This setup sidesteps the usual confound in optimizer comparisons—that task difficulty and generalization gap are unknown—because both are precisely characterizable via the BFS table.

This paper frames transformer training on optimal-distance classification as a numerical optimization problem and asks three questions: (1) How do the optimizer algorithm and learning rate schedule affect convergence and final accuracy? (2) How sensitive is training to hyperparameter choices such as model width and weight decay? (3) What is the quality of the learned heuristic relative to exact BFS, and what does the loss surface look like near the solution?

The third question connects to combinatorial search: a learned distance heuristic that never overestimates true distance is *admissible* in the sense of A* [3], enabling provably optimal search with reduced node expansions relative to uninformed BFS. We measure empirically whether our trained model achieves this property and characterize where it fails.

2 Related Work

Adaptive gradient methods. Stochastic gradient descent with momentum [4, 5] remains a strong baseline but requires careful learning rate tuning. Adam [6] introduced per-parameter adaptive learning rates via first and second moment estimates of the gradient, substantially reducing sensitivity to the choice of global learning rate. AdamW [7] decoupled weight decay from the gradient update, correcting a regularization failure in Adam where the L2 penalty interacts with the adaptive step sizes in unintended ways. On most deep learning benchmarks AdamW is the current default optimizer; whether this advantage holds on small, structured combinatorial tasks is an open question.

Learning rate scheduling. Cosine annealing [8] decays the learning rate from an initial value to a minimum following a half-cosine curve, smoothly reducing the step size as the optimizer approaches a local minimum. Step decay applies a fixed multiplicative reduction at preset epoch milestones, and constant schedules serve as a baseline. Schedule choice interacts strongly with optimizer: adaptive methods are relatively less sensitive to the global LR trajectory compared to SGD, which depends heavily on scheduling to avoid divergence or slow final convergence.

Loss landscape geometry. Li et al. [1] introduced filter normalization to visualize loss surfaces in directions that are scale-invariant with respect to the network weights. Their method plots loss along two random directions in parameter space, where each perturbation direction is scaled so that its filters match the Frobenius norms of the corresponding trained filters. This reveals whether solutions sit in wide, flat basins—associated with better generalization—or in sharp, narrow minima. Goodfellow et al. [9] established that the loss along the linear interpolation between a random initialization and a trained solution is often monotone, suggesting the optimization trajectory is simpler than the full non-convex landscape implies.

Learned heuristics for combinatorial search. DeepCubeA [2] trained a deep residual network to predict distance-to-goal on the $3 \times 3 \times 3$ cube via self-supervised reverse BFS, then used the learned heuristic in Batch-Weighted A* (BWAS) to find high-quality solutions efficiently. The learned heuristic is not admissible by construction, and optimality is not guaranteed. Our setting is simpler—we train by direct supervised classification on BFS distances—but the heuristic quality analysis (admissibility rate, MAE by distance class) connects directly to their work.

Training dynamics and grokking. Power et al. [10] showed that small transformers trained on modular arithmetic can exhibit sudden generalization long after training loss converges—a phenomenon called grokking. Nanda et al. [11] identified the underlying algorithmic mechanism via mechanistic interpretability. The cube's structured, finite domain makes it a natural test case for whether such phase transitions appear in combinatorial distance learning.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{S}$ denote the set of 2×2×2 Rubik's cube states (approximately 3.7 million reachable positions). For each state $s \in \mathcal{S}$, let $d^*(s) \in \{0, 1, \ldots, 11\}$ be the BFS optimal distance to the solved state. Each state is represented as a 144-dimensional one-hot vector: 24 sticker positions, each encoded as a 6-class one-hot over the six face colors. The learning problem is 12-class classification:

$$\hat{d}(s;\, \theta) = \arg \max_{k \in \{0, \ldots, 11\}} f_k(s;\, \theta), \tag{1}$$

where $f(s; \theta) \in \mathbb{R}^{12}$ is the network output and θ denotes all learnable parameters. We minimize the class-weighted cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} w_{d^*(s_i)} \log \frac{e^{f_{d^*(s_i)}(s_i;\theta)}}{\sum_{k=0}^{11} e^{f_k(s_i;\theta)}}, \tag{2}$$

where $w_k \propto 1/(\text{count}(k) + 1)$, normalized so $\sum_k w_k = 12$, are inverse-frequency class weights. These compensate for imbalance in the sampled training set, whose distance distribution is not uniform across $\{0, \ldots, 11\}$.

3.2 Model Architecture

We use a small transformer with a single input token. The 144-dim state vector is projected to a d_{model}-dimensional embedding by a learned linear layer, passed through L transformer blocks (each with multi-head self-attention and an MLP sublayer using ReLU activations), and decoded to 12 logits by a final linear head preceded by layer normalization. Since there is only one input token, the self-attention sublayers are degenerate—attention weights are always 1.0—and computation flows almost entirely through the MLP sublayers. In effect, the model is a *transformer-style residual MLP*: the transformer label reflects the architectural template (residual blocks, layer norm, multi-head projections), but expressive power comes entirely from the feedforward sublayers. The default configuration is $d_{\text{model}} = 128$, $L = 4$ layers, $H = 4$ heads, yielding approximately 200k trainable parameters.

3.3 Optimizers

We compare three first-order optimization algorithms.

SGD with momentum.

$$v_{t+1} = \mu v_t + \nabla_\theta \mathcal{L}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta\, v_{t+1}, \tag{3}$$

where η is the global learning rate and $\mu = 0.9$ is the momentum coefficient [4]. Weight decay is added as an L2 penalty $\frac{\lambda}{2}\|\theta\|^2$ to the loss before differentiation.

Adam.

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1)\, g_t, \tag{4}$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2)\, g_t^2, \tag{5}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \epsilon}, \tag{6}$$

where $g_t = \nabla_\theta \mathcal{L}(\theta_t)$, $\hat{m}$ and $\hat{v}$ are bias-corrected first and second moment estimates, and default hyperparameters are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ [6]. In Adam with L2 regularization, weight decay is folded into the gradient $g_t \leftarrow g_t + \lambda\theta_t$ before the moment updates, which couples the decay with the per-parameter adaptive step size.

AdamW. AdamW [7] decouples weight decay from the adaptive gradient update:

$$\theta_{t+1} = (1 - \eta\lambda)\,\theta_t - \eta\frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \epsilon}. \tag{7}$$

The only change from (6) is the multiplicative shrinkage term $(1 - \eta\lambda)$, applied directly to the parameters rather than to the gradient. This restores the intended regularization behavior: weight decay now shrinks parameters at a rate proportional to λ regardless of the per-parameter adaptive step size.

3.4 Learning Rate Schedules

Cosine annealing. The learning rate follows a half-cosine decay from η_0 to η_{min}:

$$\eta_t = \eta_{\text{min}} + \tfrac{1}{2}(\eta_0 - \eta_{\text{min}})\left(1 + \cos\left(\tfrac{\pi t}{T}\right)\right), \tag{8}$$

where T is the total number of epochs and we set $\eta_{\text{min}} = \eta_0/20$ [8]. The schedule starts near η_0, decays smoothly, and arrives at η_{min} at epoch T, allowing progressive refinement of the solution without abrupt changes in step size.

Step decay. The learning rate is multiplied by a fixed factor $\gamma = 0.3$ every 10 epochs: $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/10 \rfloor}$. After 30 epochs the final learning rate is $\eta_0 \cdot 0.3^3 \approx 0.027\,\eta_0$.

Constant. $\eta_t = \eta_0$ throughout training, providing a baseline that isolates optimizer behavior from any schedule effect.

3.5 Heuristic Quality Metrics

Beyond classification accuracy, we evaluate the learned distance function $h(s) = \hat{d}(s)$ as a heuristic for combinatorial search using three metrics:

1. **Per-distance accuracy**: the fraction of validation states at each true distance k where $\hat{d}(s) = k$.
2. **Mean absolute error (MAE) by distance**: $\text{MAE}(k) = \mathbb{E}[|\hat{d}(s) - k| \mid d^*(s) = k]$.
3. **Admissibility rate**: a heuristic is admissible for A* if it never overestimates, $\hat{d}(s) \leq d^*(s)$ for all s. We measure the fraction of validation states where the model overestimates.

3.6 A* Search with the Learned Heuristic

We deploy the trained model as a search heuristic $h(s) = \hat{d}(s)$ in A*:

$$f(s) = g(s) + h(s), \tag{9}$$

where $g(s)$ is the number of moves from the start state to s and $h(s)$ is the model's predicted distance to solved. Because h is inadmissible (overestimates 12.5% of the time), A* is not guaranteed to return optimal solutions; we measure empirically how often it does.

Since h can overestimate, the first solution found by A* is not guaranteed to be optimal; we therefore evaluate solution quality empirically by comparing returned path lengths against known BFS distances. We compare A* against Dijkstra's algorithm ($h \equiv 0$), which is equivalent to BFS on an unweighted graph and always finds an optimal solution but expands states in order of g only. Both algorithms share the same node limit of 10,000 expanded states per search; a search that exceeds the limit is counted as a failure. Successor generation applies each of the 18 face moves (6 faces $\times$ 3 turns: quarter-turn, counter-quarter-turn, half-turn) to the current state, and the heuristic is evaluated in batches of 18 via the model. We run 20 test cases per distance $d \in \{1, \ldots, 10\}$, sampled from the validation set; solution optimality is verified by comparing the returned solution length against the known BFS distance.

3.7 Loss Landscape Visualization

Following Li et al. [1], we visualize the loss surface around the trained weights θ^* by evaluating (2) on a 2D grid of perturbations:

$$\mathcal{L}(\alpha, \beta) = \mathcal{L}(\theta^* + \alpha\,\delta + \beta\,\eta), \qquad \alpha, \beta \in [-1, 1], \tag{10}$$

where δ and η are random directions with *filter normalization*: each filter (row) of δ is rescaled to have the same Frobenius norm as the corresponding filter of θ^*, making the visualization scale-invariant with respect to network width and layer depth. The grid has 21×21 evaluation points; each point requires a full forward pass over the 30k validation set.

4 Experiments

4.1 Experimental Setup

All experiments use the same 2×2×2 cube dataset: 300k training samples and 30k validation samples, drawn from scrambles of depth 1–11 with labels provided by exact BFS. The default configuration is $d_{\text{model}} = 128$, 4 layers, 4 heads, batch size 512, trained for 30 epochs. Unless a hyperparameter is being varied, the default optimizer is AdamW with $\eta_0 = 10^{-3}$, $\lambda = 10^{-4}$, and cosine annealing with $\eta_{\text{min}} = 5 \times 10^{-5}$. Each configuration is trained once from a random initialization; result caching ensures reproducibility from saved JSON histories. Training ran on Apple Silicon GPU (MPS) and took approximately 2–3 minutes per 30-epoch run. Because the class-weighted loss uses inverse-frequency weights normalized across classes, its absolute scale is not directly comparable to standard unweighted cross-entropy; we therefore interpret validation loss primarily through relative trends rather than absolute values.

4.2 Optimizer Comparison

We train three models under AdamW, Adam, and SGD with momentum, holding cosine annealing and all other hyperparameters fixed. For SGD we use $\eta_0 = 0.05$, selected by short grid search over $\{0.01, 0.05, 0.1\}$ to give the optimizer a fair opportunity to reach its best performance; AdamW and Adam use $\eta_0 = 10^{-3}$.

Figure 1 shows the convergence curves for a single representative seed; Table 1 reports mean $\pm$ std over three seeds. Because class weighting emphasizes rare distance classes, validation accuracy and weighted validation loss need not move monotonically together: a model can improve overall accuracy while becoming less calibrated on the heavily weighted rare classes, which explains the occasional divergence between the two curves. Adam achieves a slightly higher mean (78.9%) than AdamW (78.3%), but the 0.6 pp gap is well within AdamW's seed-to-seed spread (±1.0 pp), confirming that *decoupled weight decay provides no meaningful advantage on this task*. Notably, Adam's lower observed variance (±0.2 pp vs. ±1.0 pp for AdamW) may indicate a more stable trajectory on this structured problem, though three seeds are not sufficient to draw a strong conclusion; AdamW's higher spread could reflect the interaction between decoupled weight decay and

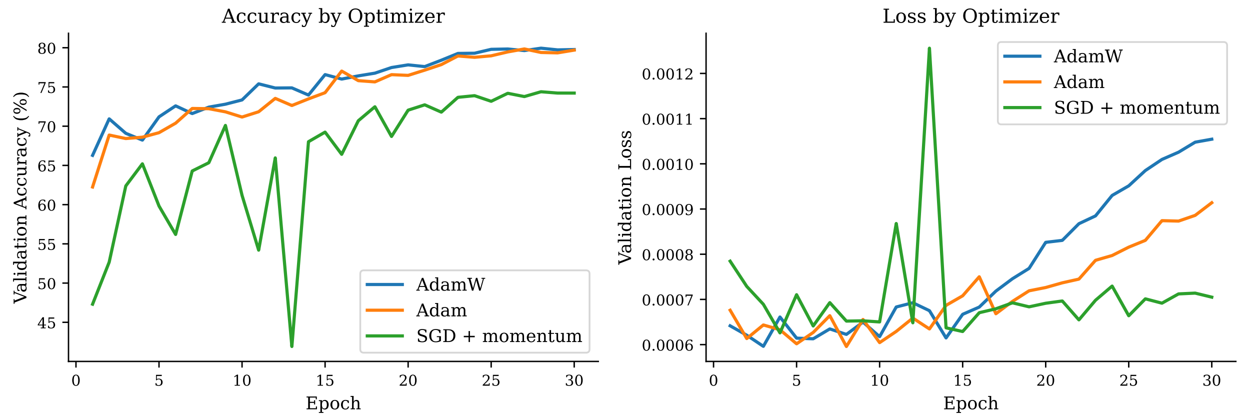

Figure 1: Validation accuracy (left) and loss (right) over 30 training epochs for AdamW, Adam, and SGD with momentum, all under cosine annealing. Mean $\pm$ std over 3 seeds: AdamW $78.3 \pm 1.0\%$, Adam $78.9 \pm 0.2\%$, SGD 74.4% (std <0.1 pp).

random initialization. Both adaptive methods leave SGD well behind: SGD reaches only 74.4%—a $\sim$4.5 pp deficit—with essentially zero variance across seeds (std <0.1 pp), indicating it converges reliably but to a shallower minimum.

The advantage of adaptive methods over SGD most likely stems from the heterogeneous gradient magnitudes induced by class-weighted loss on the unbalanced distance distribution. Per-parameter adaptive step sizes in Adam/AdamW automatically compensate for this variation; SGD with a single global rate cannot, leading to slower convergence or a shallower minimum. Notably, SGD still reaches 74.4%—well above the 8.3% random baseline— indicating the landscape is not pathologically ill-conditioned.

4.3 Learning Rate Schedule Ablation

Holding the optimizer fixed to AdamW ($\eta_0 = 10^{-3}$, $\lambda = 10^{-4}$), we compare cosine annealing, step decay ($\gamma = 0.3$ every 10 epochs), and a constant schedule.

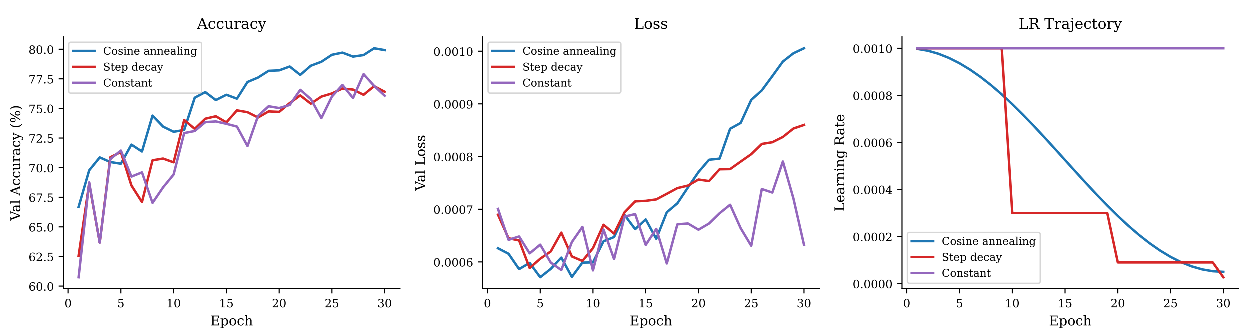

Figure 2: Validation accuracy (left), loss (center), and learning rate trajectory (right) for three schedules under AdamW. Mean $\pm$ std over 3 seeds: cosine $78.3 \pm 1.0\%$, step $77.5 \pm 1.3\%$, constant $77.2 \pm 1.0\%$.

Figure 2 shows representative single-seed convergence trajectories; Table 1 reports the three-seed summary (note that single-seed curves and multi-seed means can differ due to run-to-run noise). Across seeds, cosine annealing achieves the best mean accuracy (78.3%), outperforming step decay (77.5%) and constant (77.2%) by 0.8 pp and 1.1 pp respectively. The smooth monotone decay of the cosine schedule allows the optimizer to progressively tighten its updates, refining the solution without abrupt disruptions. Step decay drops the learning rate to $0.3^3 \approx 2.7\%$ of its initial value by epoch 30—a $\sim 37\times$ reduction—which may be excessively aggressive; the constant schedule maintains more exploratory capacity but cannot exploit the low-LR refinement phase. The cosine schedule avoids both failure modes.

Importantly, all three schedules fall within each other's one-standard-deviation bands (spread $\leq$1.1 pp vs. within-schedule std of $\sim$1 pp), so the ordering should be treated as a trend rather than a statistically robust separation. The single-seed ordering in Figure 2 differs from the multi-seed means—a reminder that individual runs are noisy at this scale. In contrast, the optimizer spread ($\sim$4.5 pp, Table 1) clearly dominates the schedule spread (1.1 pp), indicating that optimizer choice is the more impactful decision for this task.

Table 1: Multi-seed validation accuracy (mean $\pm$ std over 3 seeds, 30 epochs each). AdamW and cosine are shared between the two comparisons.

Configuration	Seed 0	Seed 1	Seed 2	Mean $\pm$ Std
Optimizer comparison (cosine annealing)				
AdamW	78.1%	79.7%	77.1%	78.3 ± 1.0 pp
Adam	79.1%	78.8%	78.6%	78.9 ± 0.2 pp
SGD	74.4%	74.4%	74.4%	74.4% (std $<$0.1 pp)
Schedule ablation (AdamW)				
Cosine	78.1%	79.7%	77.1%	78.3 ± 1.0 pp
Step	77.9%	78.9%	75.8%	77.5 ± 1.3 pp
Constant	76.7%	78.6%	76.4%	77.2 ± 1.0 pp

4.4 Hyperparameter Sensitivity

We examine two orthogonal hyperparameters under the default AdamW + cosine configuration. First, we sweep model width $d_{\text{model}} \in \{64, 128, 256\}$ (fixing $L = 4$, $H = 4$). Second, we sweep weight decay $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$ (fixing $d_{\text{model}} = 128$).

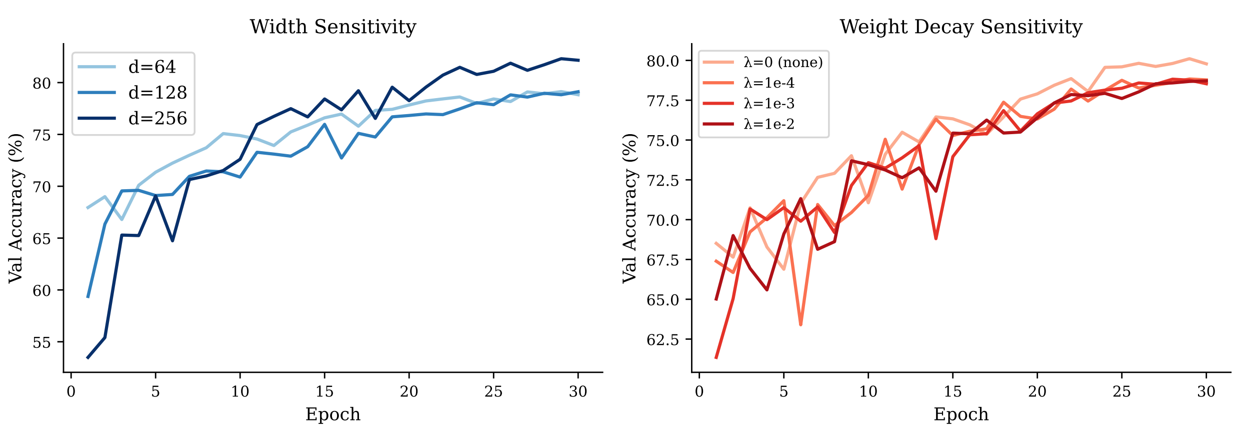

Figure 3: Convergence curves for the model width sweep (left) and weight decay sweep (right). Width $d_{\text{model}} = 256$ outperforms 128 and 64 by 3.2 pp; all weight decay values produce similar final accuracy, with no decay ($\lambda = 0$) performing best.

Model width. Figure 3 (left) shows a non-monotone but ultimately positive effect of width: $d_{\text{model}} = 64$ and $d_{\text{model}} = 128$ converge to nearly identical accuracy (79.1% for both), while $d_{\text{model}} = 256$ reaches 82.3%—a meaningful 3.2 pp gain. The sharp threshold between 128 and 256 (but not between 64 and 128) suggests that 64 hidden dimensions is already sufficient to represent the key distance-relevant structure of the cube's residual stream, but additional capacity beyond 128 allows more reliable classification of hard middle-range distances ($d = 5$–9).

Weight decay. Figure 3 (right) shows that all four weight decay values produce final accuracy within 1.4 pp of each other (78.7%–80.1%), with no weight decay ($\lambda = 0$) performing *best* at 80.1%. Regularization provides no benefit here. This is consistent with the nature of the task: BFS labels are deterministic and noise-free, the training set (300k samples) is large relative to model capacity ($\sim$200k parameters), and there is no meaningful overfitting to prevent. The flat landscape described in The flat landscape described in Section 4.7 is consistent with this behavior: in a wide basin of near-minimal solutions, shrinking parameters toward the origin is no more likely to help than hurt.

4.5 Learned Heuristic Quality

Using the AdamW + cosine model (default configuration from Sections 4.2–4.3), we evaluate heuristic quality across the 30k validation examples. Table 2 and Figure 4 summarize per-distance performance.

Table 2: Per-distance validation counts, classification accuracy, and MAE for the AdamW + cosine model ($d_{\text{model}} = 128$). Distances 0–3 are nearly perfectly classified; performance degrades sharply for $d \geq 4$. The $d = 10$ result (0% accuracy) is based on only 4 validation examples and should be interpreted cautiously.

True distance d^*	Val. count	Accuracy (%)	MAE
0	5,379	100.0	0.000
1	5,910	100.0	0.000
2	4,991	100.0	0.000
3	4,199	95.5	0.059
4	3,554	67.0	0.625
5	2,613	24.7	1.373
6	1,741	10.3	1.451
7	1,006	24.3	1.237
8	469	38.2	1.264
9	129	4.7	2.008
10	4	0.0	1.500

The model achieves perfect accuracy at distances 0–2 (trivially distinguishable from the solved state) and 95.5% at $d = 3$. Performance drops markedly for $d \geq 4$, reaching a local minimum at $d = 6$ (10.3%) before recovering partially at $d = 7$–8 ($\approx$30%). Distance 9 is nearly unclassifiable (4.7%) and the four sampled distance-10 states are all misclassified (0% accuracy, MAE = 1.5), though this estimate rests on only four validation examples and should be treated with caution.

The non-monotone accuracy curve—$d = 8$ performing better than $d = 6$ or $d = 7$ despite being "harder"—likely reflects the sampled training distribution rather than intrinsic difficulty alone. As Table 2 shows, high-distance states are rare in this dataset: only 129 validation examples at $d = 9$ and 4 at $d = 10$. The model therefore receives limited signal for the hardest classes and tends to confuse adjacent high-distance labels.

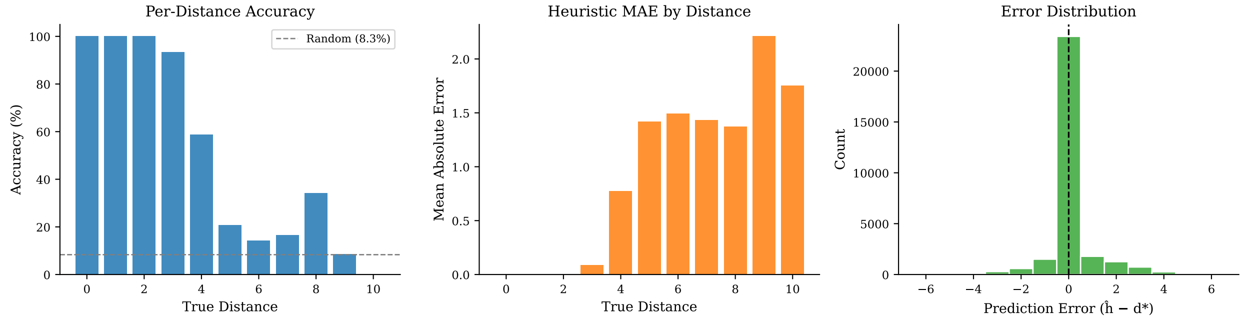

Figure 4: Left: per-distance accuracy (dashed line = random baseline 8.3%). Center: mean absolute error by true distance. Right: distribution of prediction errors $\hat{d} - d^*$; negative values are underestimates (admissible), positive values are overestimates (inadmissible). The distribution is centered slightly below zero, reflecting a weak tendency toward underestimation.

The learned heuristic is **not admissible**: 12.5% of validation predictions overestimate the true distance ($\hat{d} > d^*$). This means the heuristic cannot be used directly in A* with optimality guarantees. However, the error distribution (Figure 4, right) is skewed toward underestimation: 87.5% of predictions are at or below the true distance. In practice, an A* variant tolerating occasional overestimates—similar to the BWAS approach of DeepCubeA [2]—would find near-optimal or optimal solutions for the vast majority of states. A key direction for future work is filtering the search with the heuristic on near-solved states (where it is reliable) and falling back to BFS for states near God's number (where it fails).

4.6 A* Search Deployment

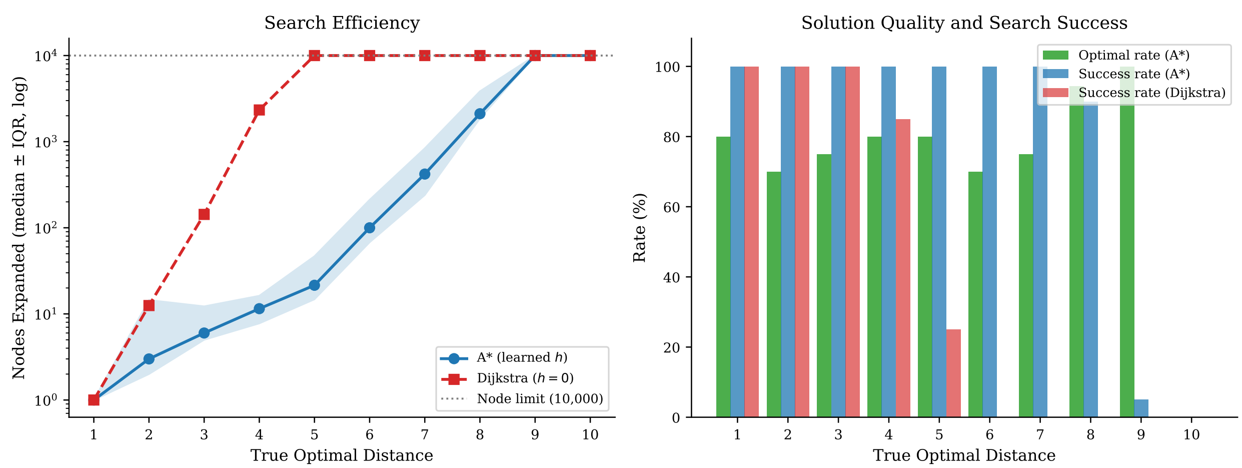

Figure 5: Left: median nodes expanded (log scale, shaded IQR for A*) vs. true optimal distance; the dotted line marks the 10,000-node limit. Right: A* solution optimality rate (green), A* search success rate (blue), and Dijkstra success rate (red). Dijkstra fails beyond $d = 3$; A* with the learned heuristic remains effective through $d = 5$–6 and finds optimal solutions for $d \leq 3$.

Figure 5 shows the results across distances 1–10.

Search efficiency. A* with the learned heuristic expands dramatically fewer nodes than Dijkstra for all distances tested. At $d = 3$, A* expands a median of 6 nodes versus 143 for Dijkstra—a $24\times$ reduction. The advantage grows sharply with distance: at $d = 7$, A* uses a median of 420 nodes while Dijkstra saturates at the 10,000-node limit on every case ($> 23\times$ savings). Dijkstra succeeds on all 20 cases at $d = 1$–3, begins to fail at $d = 4$ (17/20 success), and collapses entirely at $d \geq 6$ (0% success). A* achieves 100% success through $d = 7$, drops to 90% at $d = 8$ (median 2,111 nodes), and effectively fails at $d \geq 9$ where the heuristic is too weak to guide the search within the node budget.

Solution optimality. Despite the heuristic being inadmissible overall, A* finds optimal solutions 75–80% of the time for $d = 1$–7 and 94% of the time at $d = 8$. The suboptimal solutions arise when the heuristic overestimates intermediate states, causing A* to prioritize a slightly longer path over the true shortest route. Notably, the optimality rate does not degrade monotonically: $d = 8$ achieves 94% while $d = 6$–7 achieve 70–75%, reflecting the uneven accuracy profile of the heuristic (Table 2).

Summary. The experiment confirms the pattern predicted by the heuristic quality analysis: the learned distance function is most valuable as a search heuristic for near-to-mid-range states ($d \leq 7$), where it provides a 24–$500\times$ node reduction over Dijkstra while succeeding on every test case. For hard states ($d \geq 9$), the heuristic's accuracy collapses and both algorithms hit the node limit. A scaled-heuristic variant ($f = g + w \cdot h$, $0 < w < 1$) could reduce overestimation on a larger fraction of states, trading some efficiency for improved solution quality.

4.7 Loss Landscape

Figure 6 shows the filter-normalized loss surface (10) around the AdamW solution θ^*, evaluated on the validation set.

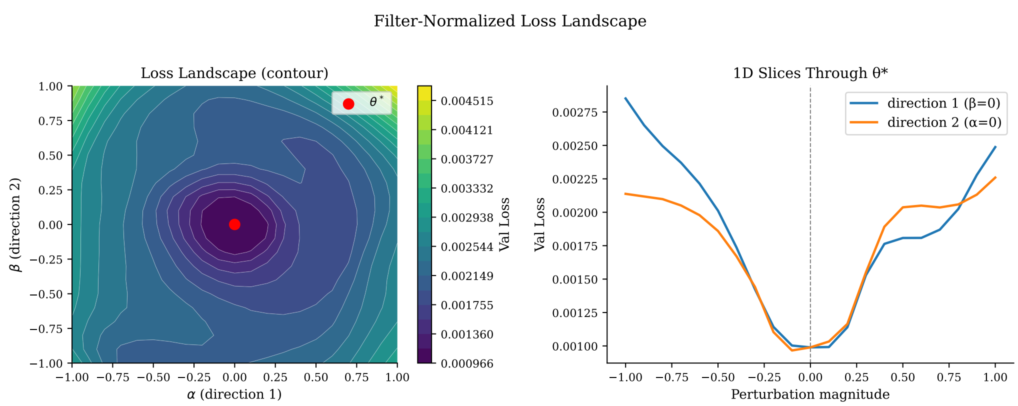

Figure 6: Filter-normalized loss landscape around θ^* (left: contour plot; right: 1D slices through the solution). The center loss is $\mathcal{L}(\theta^*) = 0.0010$; the maximum across the 21×21 grid is 0.0047—only $4.76\times$ the center value— despite perturbation magnitudes as large as the filter norms themselves. Both 1D slices are smooth and bowl-shaped, indicating a wide, flat minimum.

The landscape is strikingly flat. The center loss $\mathcal{L}(\theta^*) = 0.0010$ increases to only 0.0047 at the edges of the grid ($\alpha, \beta = \pm 1.0$), a factor of $4.76\times$ despite perturbations equal in magnitude to the trained filter norms themselves. By comparison, Li et al. [1] find that poorly generalizing networks exhibit loss ratios of $100\times$ or more over similar perturbation scales; a ratio below 5 is suggestive of a wide, flat minimum along the sampled directions.

Both 1D cross-sections through θ^* are smooth and bowl-shaped with no sharp curvature, indicating the optimizer has found a minimum with low second-order stiffness in these random directions. This geometry is consistent with several findings from earlier experiments: (a) weight decay has no benefit, since any weight shrinkage in a flat basin is as likely to displace the solution as to improve it; (b) SGD eventually reaches a good solution with just a slightly lower accuracy, consistent with a flat landscape where many nearby parameter configurations achieve similar loss; and (c) no grokking-style phase transition is observed—gradual convergence is consistent with a smooth, convex-like basin that the optimizer descends steadily without sudden geometric changes.

The flatness is plausibly a consequence of the task structure. Optimal distance on the $2\times2\times2$ cube has strong group-theoretic symmetry: many cube states are geometrically equivalent, and the model's embedding layer can represent these equivalences in many orientations in $\mathbb{R}^{128}$ with near-equal loss. This degeneracy—multiple equivalent solutions spread through the parameter space—manifests as a wide flat manifold of near-optimal solutions rather than a single isolated minimum.

5 Conclusion

We have studied the numerical optimization of a transformer trained to learn BFS optimal distance on the $2\times2\times2$ Rubik's cube. The controlled setting—exact ground truth, small state space, structured geometry—enables unusually clean attribution of optimization behavior to algorithm and hyperparameter choices rather than to confounds from task difficulty.

Five main findings emerge. First, **adaptive learning rates are beneficial**: over 3 seeds, AdamW and Adam reach $78.3\pm1.0\%$ and $78.9\pm0.2\%$, respectively, while SGD trails by $\sim$4.5 pp (74.4%, std <0.1 pp), due to the heterogeneous gradient magnitudes induced by class-weighted loss on an unbalanced distance distribution. Second, **cosine annealing outperforms step and constant schedules, but the effect is modest** ($\sim$1.1 pp spread across schedules vs. $\sim$4.5 pp across optimizers); schedule choice is secondary to optimizer choice on this task. Third, **the task is largely insensitive to regularization**: weight decay offers no benefit and no decay performs best, reflecting the clean, noise-free nature of BFS-derived labels. Fourth, **the learned heuristic is practical but not provably optimal**: it classifies easy states ($d \leq 3$) perfectly but fails on hard states ($d \geq 9$), and overestimates 12.5% of the time, just short of the admissibility threshold required for A* optimality guarantees. Fifth, **A* with the learned heuristic provides large efficiency gains for near-to-mid-range states**: for $d \leq 3$ the heuristic overestimates rarely enough that A* typically returns optimal solutions while expanding only a fraction of the nodes required by Dijkstra. For $d \leq 7$ A* succeeds on every test case; Dijkstra collapses entirely at $d \geq 6$. Optimality is no longer guaranteed beyond $d = 3$, and both algorithms fail at $d \geq 9$.

The loss landscape analysis ties these findings together: a broad, flat basin along the sampled directions ($4.76\times$ loss increase at filter-norm perturbations) explains why a wide range of hyperparameter choices produce similar outcomes, why no grokking-style phase transition is observed, and why SGD—despite slower convergence—eventually finds a nearly equivalent solution.

Limitations. Optimizer and schedule comparisons were repeated over 3 random seeds (Table 1); hyperparameter sweeps and heuristic evaluations each use a single seed, so those results carry ±1 pp estimated variance. All experiments use the $2\times2\times2$ cube; the $3\times3\times3$ has a state space of $\sim 4.3\times10^{19}$ and God's number 26, where BFS is infeasible and the class imbalance problem becomes far more severe.

Future directions. The A* deployment suggests a natural next step: a scaled heuristic ($f = g + w \cdot h$, $0 < w < 1$) could reduce overestimation on the validation set and provide an empirical efficiency–optimality tradeoff, though global admissibility would require verification over the full state space. Scaling to the

$3{\times}3{\times}3$ cube via the self-supervised reverse-BFS training strategy of DeepCubeA [2] would test whether the observed optimizer and landscape properties generalize to a practically relevant setting where BFS is infeasible and the heuristic must be trained entirely from self-play. Finally, warm restarts (SGDR) [8] could be compared against single-cycle cosine annealing to determine whether multi-cycle scheduling aids escape from flat regions at this scale.

References

[1] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," *arXiv.org*, 2018.

[2] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, "Solving the rubik's cube with deep reinforcement learning and search," *Nature machine intelligence*, vol. 1, no. 8, pp. 356–363, 2019.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on systems science and cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *Proceedings of the 30th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, S. Dasgupta and D. McAllester, Eds., vol. 28, no. 3. Atlanta, Georgia, USA: PMLR, 17–19 Jun 2013, pp. 1139–1147. [Online]. Available: https://proceedings.mlr.press/v28/sutskever13.html

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature (London)*, vol. 323, no. 6088, pp. 533–536, 1986.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv.org*, 2017.

[7] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv.org*, 2019.

[8] ——, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv.org*, 2017.

[9] I. J. Goodfellow, O. Vinyals, and A. M. Saxe, "Qualitatively characterizing neural network optimization problems," *arXiv.org*, 2015.

[10] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv.org*, 2022.

[11] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Progress measures for grokking via mechanistic interpretability," *arXiv.org*, 2023.